# Reporte Técnico de Actividades Práctico-Experimentales Nro. 06

## 1. Datos de Identificación del Estudiante y la Práctica

| | |
|---|---|
| **Nombre del estudiante** | Cael Alejandro Soto Castillo |
| **Asignatura** | Simulación |
| **Ciclo** | 5 |
| **Unidad** | U2: Generación de números pseudoaleatorios y generación de variables aleatorias |
| **Resultado de aprendizaje de la unidad** | R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.. |
| **Práctica Nro.** | 06 |
| **Título de la Práctica** | Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov |
| **Nombre del Docente** | José O. Guamán Q. |
| **Fecha** | 05 de noviembre de 2026<br>06 de noviembre 2026 |
| **Horario** | Jueves 10h30 a 11h30<br>Viernes 07h30 a 09h30 |
| **Lugar** | Aula |
| **Tiempo planificado en el Sílabo** | 3 |

## 2. Objetivo(s) de la Práctica

Evaluar la calidad estadística de una secuencia de números pseudoaleatorios mediante pruebas de bondad de ajuste, utilizando los métodos Chi-cuadrado y Kolmogorov-Smirnov para determinar si los valores generados presentan un comportamiento compatible con una distribución uniforme en el intervalo $[0, 1]$.

1. Generar secuencias de números pseudoaleatorios utilizando el método congruencial lineal (LCG).
2. Aplicar la prueba estadística Chi-cuadrado para evaluar la uniformidad de las frecuencias observadas.
3. Aplicar la prueba Kolmogorov-Smirnov para comparar la distribución empírica con la distribución uniforme teórica.
4. Interpretar los estadísticos de prueba y sus respectivos valores $p$.
5. Comparar el comportamiento estadístico de diferentes configuraciones del generador pseudoaleatorio.

## 3. Materiales y reactivos

- Computador.

- Lenguaje de programación Python.
- Lenguaje de programación Java.
- Entorno de desarrollo integrado.
- Generador de números pseudoaleatorios.

# 4. Equipos y herramientas

- Python.
- NumPy.
- Matplotlib.
- Pandas.
- JDK.
- IntelliJ IDEA, Eclipse o NetBeans.
- Visual Studio Code.

# 5. Procedimiento / Metodología Ejecutada

## 5.1. Contexto de la práctica

En simulación, los números aleatorios que se usan en realidad los produce un algoritmo determinista, por eso se llaman pseudoaleatorios. Antes de confiar en ellos conviene comprobar que se comportan como se espera. Una propiedad importante es la uniformidad: que los valores se repartan de forma pareja en el intervalo $[0, 1]$. Para evaluarla, la ficha plantea esta hipótesis, con un nivel de significancia $\alpha = 0{,}05$:

$$H_0 : X \sim U(0,1) \qquad H_1 : X \not\sim U(0,1)$$

$H_0$ dice que los números siguen una distribución uniforme y $H_1$ que no. En esta práctica generé los números con un generador congruencial lineal (LCG), que calcula cada valor a partir del anterior, y les apliqué dos pruebas: Chi-cuadrado y Kolmogorov-Smirnov.

$$X_{n+1} = (aX_n + c) \bmod m, \qquad U_{n+1} = \frac{X_{n+1}}{m}$$

Aquí $X_n$ es el valor actual (al inicio, la semilla), $a$ el multiplicador, $c$ el incremento y $m$ el módulo. Al dividir entre $m$ se obtiene un número entre 0 y 1.

La prueba **Chi-cuadrado** divide $[0, 1]$ en $k$ intervalos y compara cuántos números cayeron en cada uno ($O_i$) con cuántos se esperaban ($E_i$):

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}, \qquad gl = k - 1 = 10 - 1 = 9$$

La prueba **Kolmogorov-Smirnov** (KS) compara la distribución acumulada de los datos, $F_n(x)$, con la teórica, $F(x)$, y se queda con la mayor diferencia:

$$D = \sup_x |F_n(x) - F(x)|$$

En las dos pruebas, si $p \geq 0{,}05$ no se rechaza $H_0$; si $p < 0{,}05$ se rechaza.

## 5.2. Pasos que realicé

Seguí los tres algoritmos de la ficha, primero en Java y después en Python, con el mismo orden y los mismos nombres de pasos:

1. **Definí las configuraciones del generador.** La ficha pide definir los parámetros del generador (semilla, multiplicador, incremento y módulo), pero no da valores fijos. Elegí unos parámetros clásicos de un LCG y los probé con dos semillas distintas (12345 y 54321). Como el quinto objetivo pide comparar configuraciones, agregué otras dos: RANDU y una con parámetros inadecuados ($m = 16$). Todas generan 10 000 números, que es la cantidad que indica la ficha (Tabla 2).
2. **Organicé el programa en modelo, vista y controlador** en los dos lenguajes. El modelo tiene el generador y las dos pruebas, con el cálculo de los valores $p$; la vista imprime los resultados, guarda los CSV y dibuja las gráficas; y el controlador sigue los pasos de la ficha. Los valores $p$ los calculé con fórmulas propias, sin librerías de estadística.
3. **Generé los números (Actividad 1).** Con la fórmula del LCG obtuve los 10 000 valores de cada configuración y los guardé en archivos CSV, uno por configuración y por lenguaje.
4. **Apliqué la prueba Chi-cuadrado (Actividad 2).** Dividí $[0, 1]$ en 10 intervalos, conté los números de cada uno, comparé con la frecuencia esperada (1000) y calculé $\chi^2$, los 9 grados de libertad y el valor $p$.
5. **Apliqué la prueba Kolmogorov-Smirnov (Actividad 3).** Ordené los números y, para cada uno, calculé la diferencia positiva y la negativa entre la acumulada empírica y la teórica. El mayor valor es el estadístico $D$, del que obtuve el valor $p$.
6. **Tomé la decisión estadística.** Comparé cada valor $p$ con $\alpha = 0{,}05$ y también el estadístico con su valor crítico.
7. **Comparé Python y Java.** Python lee los archivos que guardó Java y cuenta en cuántos de los 10 000 números coinciden, además de comparar $\chi^2$, $D$ y los valores $p$.
8. **Hice las gráficas** de la secuencia, las frecuencias, la curva de la distribución Chi-cuadrado, las acumuladas y los valores $p$ (Tabla 1).

El proyecto se ejecuta desde la terminal con un solo comando (`./ejecutar.sh`), que compila y ejecuta Java y después Python.

## 5.3. Gráficas del informe

**Tabla 1:** Figuras del informe, con el nombre del archivo de cada una.

| Fig. | Archivo (en `figures/`) | Contenido |
|---|---|---|
| 1 | `secuencia.png` | Primeros 100 números generados por cada configuración. |
| 2 | `dispersion.png` | Puntos $(U_n, U_{n+1})$ de cada configuración. |
| 3 | `chi_cuadrado.png` | Frecuencia observada de cada intervalo contra la esperada. |
| 4 | `chi_campana.png` | Curva de la distribución Chi-cuadrado (9 grados de libertad) con la zona de rechazo y el $\chi^2$ observado. |
| 5 | `ks_acumulada.png` | Distribución acumulada empírica contra la teórica. |
| 6 | `ks_diferencias.png` | Diferencia entre las dos acumuladas en cada punto, con $D$ observado y $D$ crítico. |
| 7 | `valores_p.png` | Valores $p$ de las dos pruebas contra $\alpha$. |

# 6. Resultados esperados

Obtener dos implementaciones funcionales que produzcan la misma secuencia cuando utilizan los mismos parámetros. El estudiante debe armar una tabla con los datos generados por ambos programas e ir comparando, además debe dar sus conclusiones.

En mi caso, esto se traduce en dos programas (uno en Python y otro en Java) que generan los mismos números con los mismos parámetros, y en tablas y gráficas para comparar lo que obtuvieron las pruebas.

# 7. Resultados

## 7.1. Generación con el LCG

Como la ficha pide definir los parámetros del generador sin fijar valores, probé cuatro configuraciones con 10 000 números cada una (Tabla 2). Las dos primeras usan los mismos parámetros con semillas distintas; la tercera es RANDU, un generador conocido, y la cuarta usa un módulo muy pequeño ($m = 16$) para ver qué pasa cuando los parámetros no son adecuados.

**Tabla 2:** Configuraciones del generador utilizadas.

| Configuración | $a$ | $c$ | $m$ | Semilla |
|---|---|---|---|---|
| Guia semilla 12345 | 1664525 | 1013904223 | $2^{32}$ | 12345 |
| Guia semilla 54321 | 1664525 | 1013904223 | $2^{32}$ | 54321 |
| RANDU | 65539 | 0 | $2^{31}$ | 1 |
| Parametros malos m=16 | 5 | 3 | 16 | 1 |

La Tabla 3 muestra los primeros diez números de la primera configuración en cada lenguaje, para comprobar que ambos programas generan lo mismo. En las cuatro configuraciones, Python y Java coincidieron en los 10 000 números. Además, en la Figura 1 se ven los primeros 100 números de cada configuración y en la Figura 2 cada número contra el siguiente para revisar a simple vista si hay algún patrón.

**Tabla 3:** Primeros 10 números con la semilla 12345 (primera configuración), generados por Python y por Java.

| $n$ | $U_n$ Python | $U_n$ Java | ¿Igual? |
|---|---|---|---|
| 1 | 0,0204026857 | 0,0204026857 | Sí |
| 2 | 0,0165478482 | 0,0165478482 | Sí |
| 3 | 0,5431557945 | 0,5431557945 | Sí |
| 4 | 0,6349040561 | 0,6349040561 | Sí |
| 5 | 0,9100295138 | 0,9100295138 | Sí |
| 6 | 0,1124616691 | 0,1124616691 | Sí |
| 7 | 0,4958894053 | 0,4958894053 | Sí |
| 8 | 0,5483488443 | 0,5483488443 | Sí |
| 9 | 0,5961001171 | 0,5961001171 | Sí |
| 10 | 0,7834504161 | 0,7834504161 | Sí |

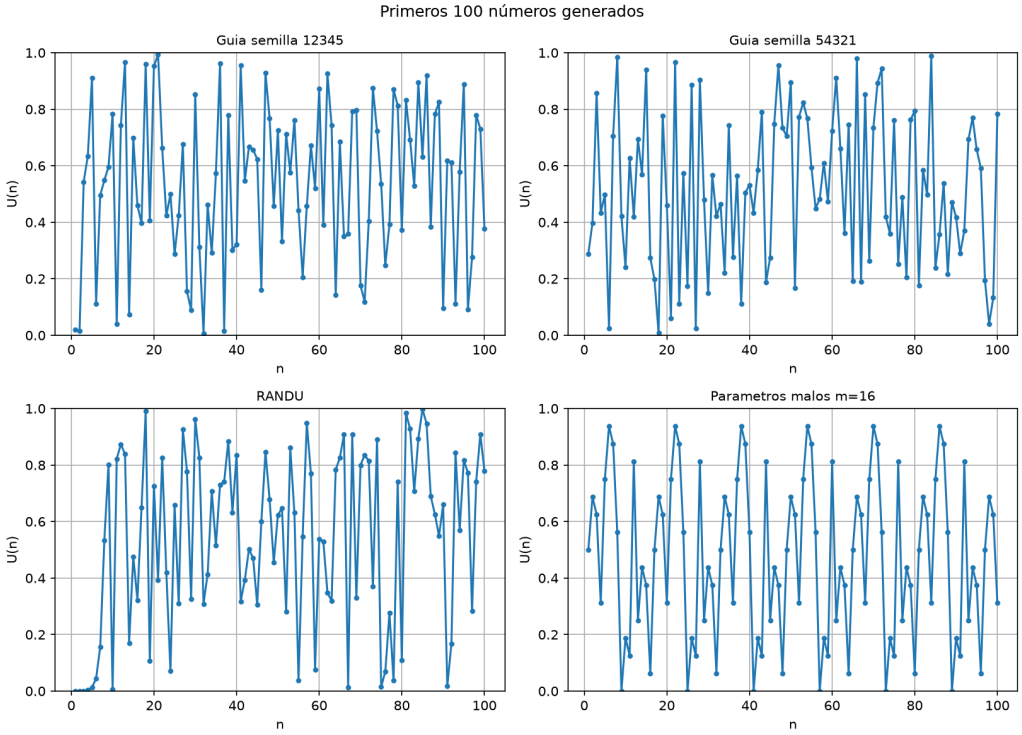



**Figura 1:** Primeros 100 números generados por cada configuración.

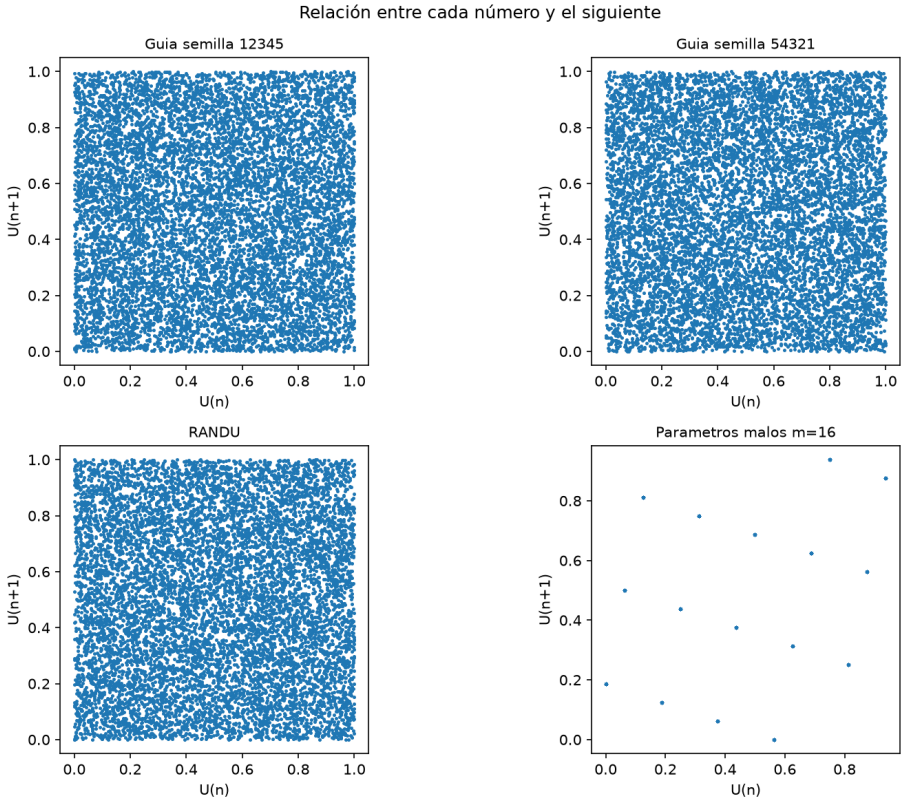



**Figura 2:** Puntos $(U_n, U_{n+1})$ de cada configuración.

## 7.2. Prueba Chi-cuadrado

Para esta prueba dividí el intervalo $[0, 1]$ en diez partes iguales. Si los números son uniformes, en cada parte deberían caer unos 1000 de los 10 000 (la frecuencia esperada).

La Tabla 4 muestra cuántos cayeron realmente en cada intervalo, y la Tabla 5 resume el estadístico Chi-cuadrado ($\chi^2$, que mide qué tan lejos están las frecuencias observadas de las esperadas), el valor $p$ y la decisión.

**Tabla 4:** Frecuencias observadas por intervalo (frecuencia esperada = 1000).

| Configuración | 0,0 | 0,1 | 0,2 | 0,3 | 0,4 | 0,5 | 0,6 | 0,7 | 0,8 | 0,9 |
|---|---|---|---|---|---|---|---|---|---|---|
| Guia semilla 12345 | 1016 | 993 | 1049 | 940 | 949 | 1063 | 1014 | 1016 | 984 | 976 |
| Guia semilla 54321 | 993 | 1023 | 1016 | 1022 | 975 | 983 | 947 | 1026 | 1034 | 981 |
| RANDU | 974 | 1011 | 984 | 1031 | 1011 | 972 | 1004 | 1064 | 959 | 990 |
| Parametros malos m=16 | 1250 | 1250 | 625 | 1250 | 625 | 1250 | 1250 | 625 | 1250 | 625 |

La primera fila de la tabla indica el inicio de cada intervalo (por ejemplo, 0,3 es el intervalo $[0{,}3;\ 0{,}4)$).

**Tabla 5:** Resultados de la prueba Chi-cuadrado ($gl = 9$, $\chi^2$ crítico $= 16{,}919$ para $\alpha = 0{,}05$).

| Configuración | $\chi^2$ | Valor $p$ | Decisión |
|---|---|---|---|
| Guia semilla 12345 | 14,160 | 0,1167 | No se rechaza $H_0$ |
| Guia semilla 54321 | 7,234 | 0,6128 | No se rechaza $H_0$ |
| RANDU | 8,812 | 0,4548 | No se rechaza $H_0$ |
| Parametros malos m=16 | 937,500 | $5{,}13 \times 10^{-196}$ | Se rechaza $H_0$ |

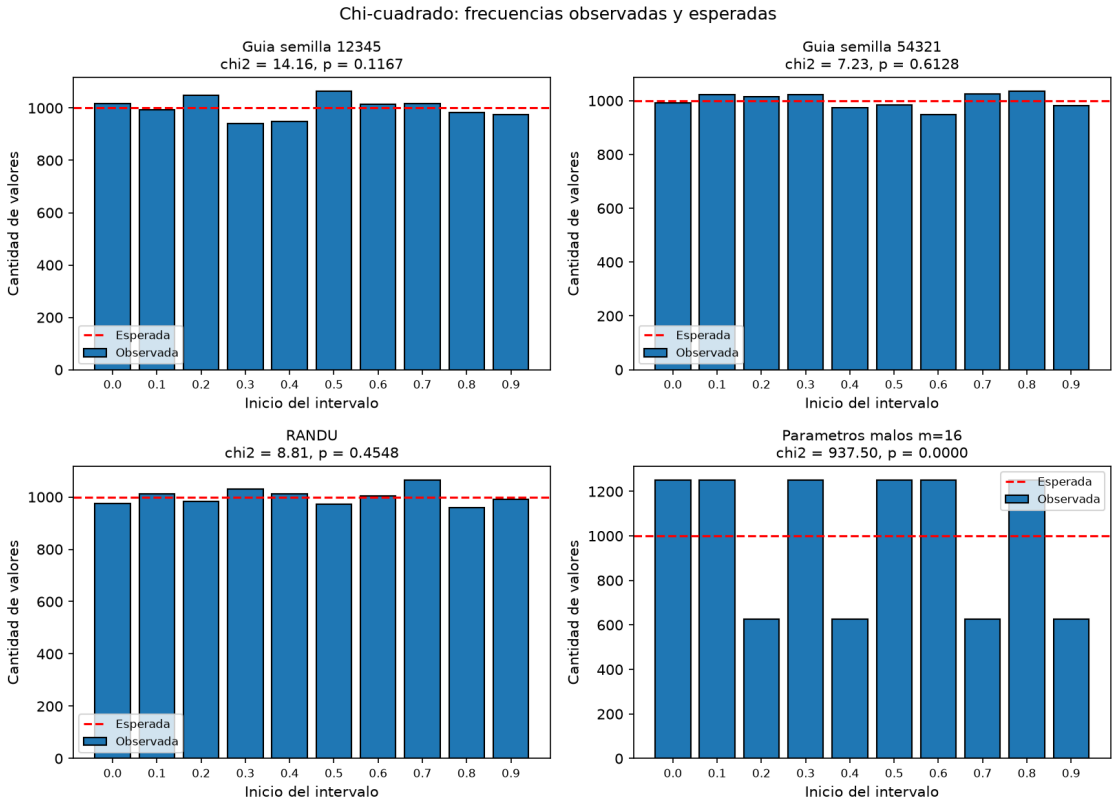



**Figura 3:** Frecuencias observadas y esperadas (línea roja) en cada intervalo.

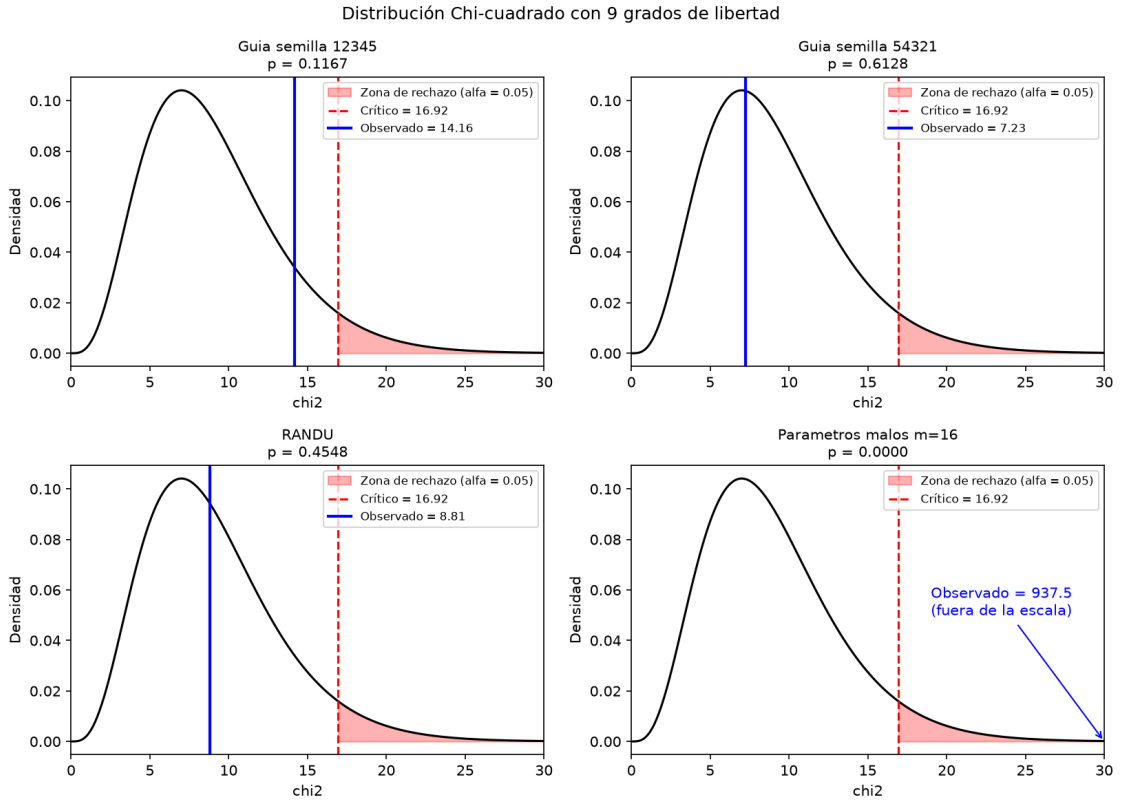



**Figura 4:** Distribución Chi-cuadrado con 9 grados de libertad. La zona roja es la región de rechazo y la línea azul el $\chi^2$ observado.

## 7.3. Prueba Kolmogorov-Smirnov

Aquí no agrupé los números en intervalos: comparé directamente la distribución acumulada de los datos con la de una uniforme. El estadístico $D$ es la mayor diferencia entre las dos, y se calcula a partir de $D^+$ y $D^-$ (la mayor diferencia por encima y por debajo de la teórica). Cuanto más pequeño es $D$, más se parece la secuencia a una uniforme (Tabla 6).

**Tabla 6:** Resultados de la prueba Kolmogorov-Smirnov ($D$ crítico $= 0{,}01357$ para $\alpha = 0{,}05$ y $n = 10\,000$).

| Configuración | $D^+$ | $D^-$ | $D$ | Valor $p$ | Decisión |
|---|---|---|---|---|---|
| Guia semilla 12345 | 0,00596 | 0,00541 | 0,00596 | 0,8686 | No se rechaza $H_0$ |
| Guia semilla 54321 | 0,00748 | 0,00524 | 0,00748 | 0,6288 | No se rechaza $H_0$ |
| RANDU | 0,00636 | 0,00509 | 0,00636 | 0,8127 | No se rechaza $H_0$ |
| Parametros malos m=16 | 0,06250 | 0,00000 | 0,06250 | $1{,}95 \times 10^{-34}$ | Se rechaza $H_0$ |

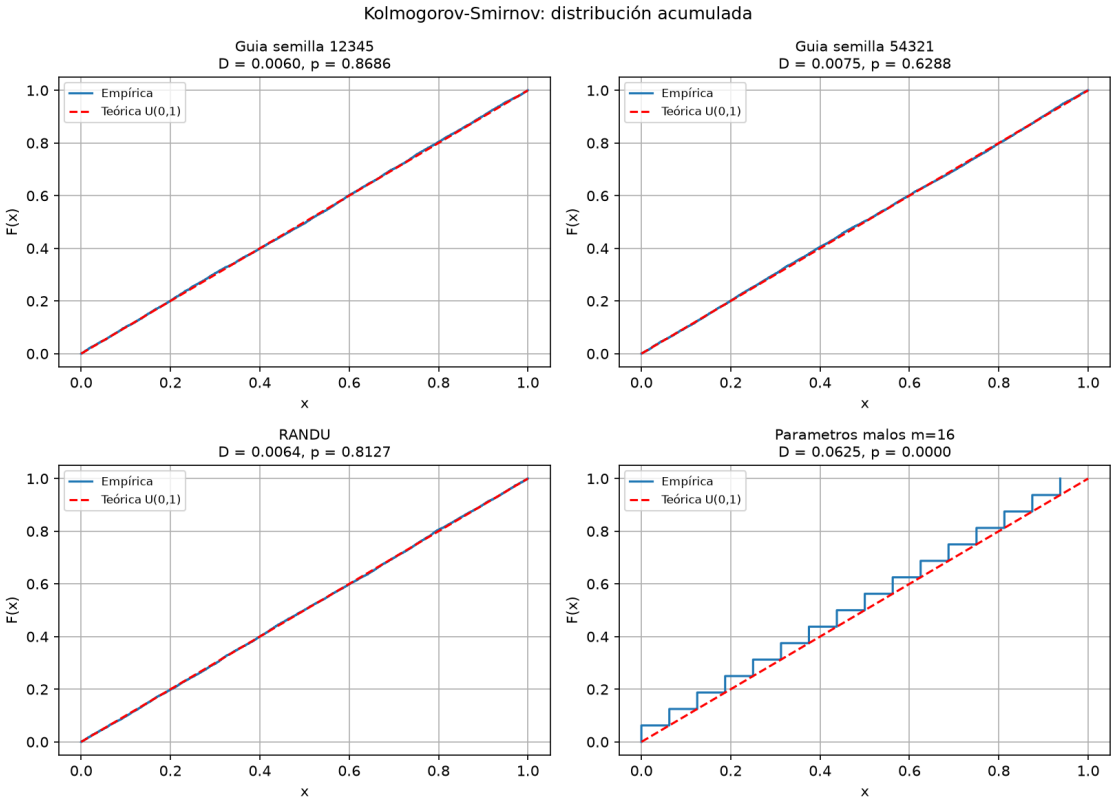



**Figura 5:** Distribución acumulada empírica (azul) contra la teórica $U(0,1)$ (línea roja).

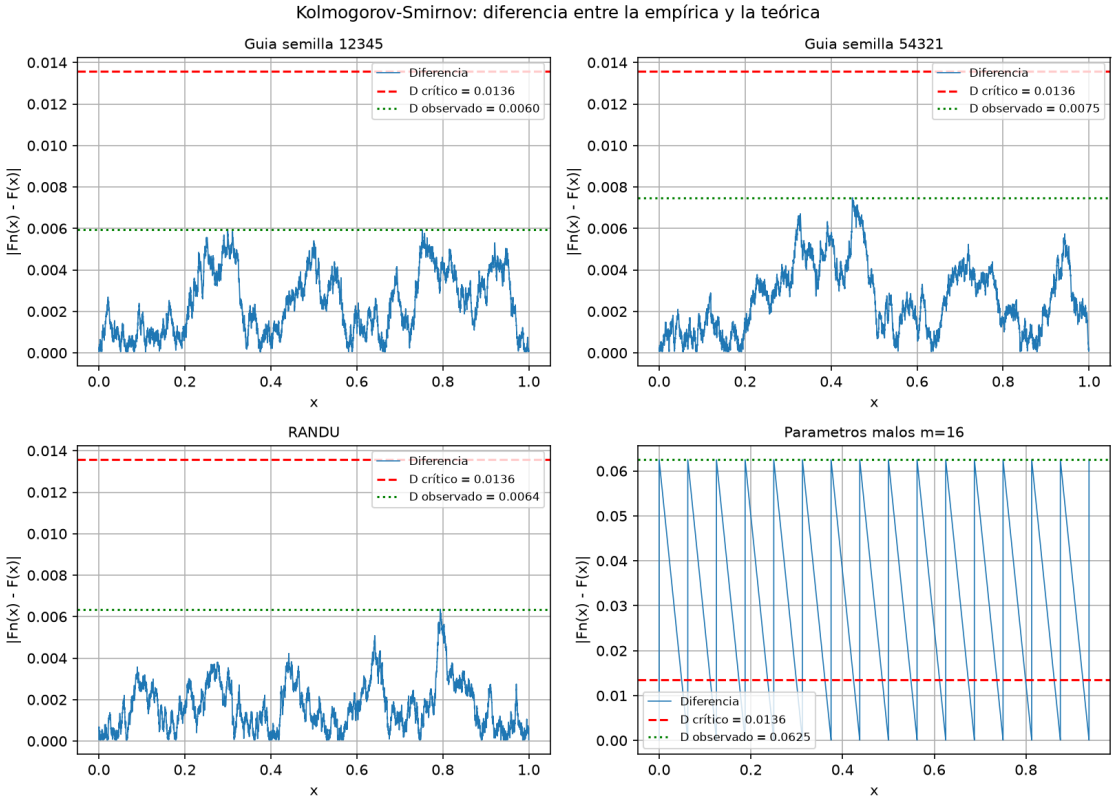



**Figura 6:** Diferencia entre la acumulada empírica y la teórica. El máximo es el estadístico $D$.

### 7.4. Comparación entre configuraciones e implementaciones

En la Tabla 7 comparé lo que obtuvieron Python y Java, y en la Figura 7 junté los valores $p$ de todas las configuraciones para ver de un vistazo cuáles se rechazan.

**Tabla 7:** Comparación de Python y Java: números iguales y coincidencia de los estadísticos.

| Configuración | Números iguales | $\chi^2$ | $p$ ($\chi^2$) | $D$ | $p$ (KS) |
|---|---|---|---|---|---|
| Guia semilla 12345 | 10 000 de 10 000 | Sí | Sí | Sí | Sí |
| Guia semilla 54321 | 10 000 de 10 000 | Sí | Sí | Sí | Sí |
| RANDU | 10 000 de 10 000 | Sí | Sí | Sí | Sí |
| Parametros malos m=16 | 10 000 de 10 000 | Sí | Sí | Sí | Sí |

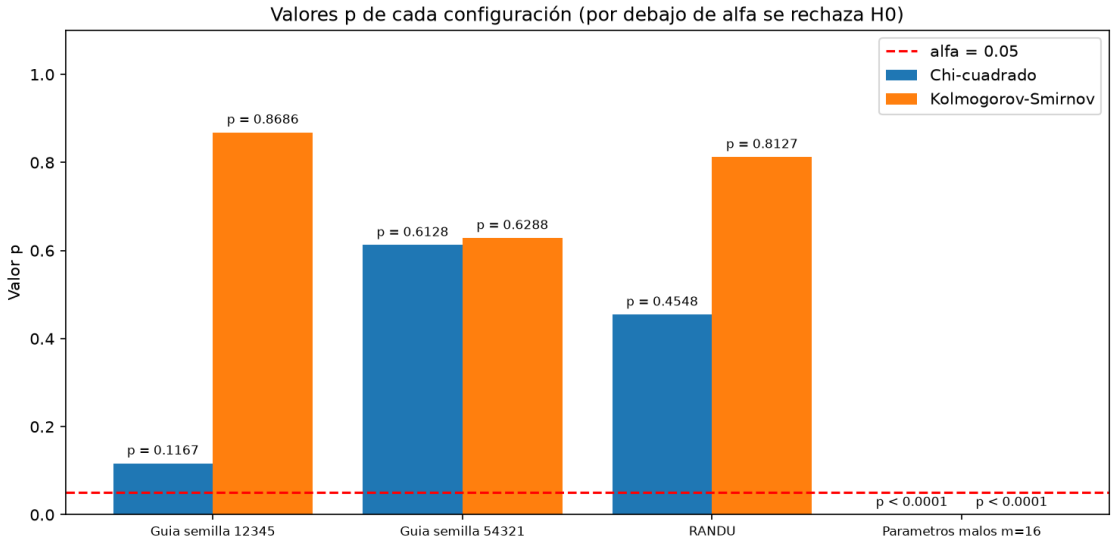



**Figura 7:** Valores $p$ de las dos pruebas para cada configuración. Por debajo de $\alpha = 0{,}05$ se rechaza $H_0$.

## 8. Análisis de Resultados

### 8.1. Configuraciones con los mismos parámetros (Tablas 5 y 6)

Empecé revisando las dos configuraciones que usan los mismos parámetros pero distinta semilla. En la prueba Chi-cuadrado, el valor $p$ (la probabilidad de obtener una diferencia como la observada si los números realmente fueran uniformes) fue de 0,1167 con la semilla 12345 y de 0,6128 con la 54321. En Kolmogorov-Smirnov fueron 0,8686 y 0,6288. Todos son mayores que el nivel de significancia $\alpha = 0{,}05$, así que en ningún caso rechacé la hipótesis nula $H_0$.

En la Figura 4 se ve esto de forma más clara: el estadístico $\chi^2$ de la primera semilla queda a la izquierda del valor crítico, es decir, fuera de la zona de rechazo, aunque más cerca de ella que el de la segunda. Esto me hizo notar que, aunque los parámetros sean los mismos, el resultado cambia entre una semilla y otra, y eso es normal porque una muestra nunca sale perfecta. Las frecuencias de la Tabla 4 se mantienen cerca de las 1000 esperadas, y en la Figura 5 la curva empírica casi se pega a la diagonal teórica. En la Figura 6 el estadístico $D$ (la mayor diferencia entre las dos curvas) queda bien por debajo de su valor crítico.

### 8.2. Comparación entre configuraciones (Figuras 2 y 7)

RANDU también pasó las dos pruebas. Esto me llamó la atención, porque es un generador que se conoce por tener problemas. Entendí que las pruebas que apliqué solo revisan cómo se reparten los valores en $[0, 1]$ y no la relación entre números consecutivos, por lo que no alcanzan a detectar sus fallas.

En cambio, la configuración con módulo pequeño ($m = 16$) se rechazó con claridad en las dos pruebas, con valores $p$ prácticamente iguales a cero (Figura 7). La Figura 2 muestra por qué: con ese módulo solo existen 16 valores distintos, así que los puntos caen en muy pocos lugares. Eso también explica las frecuencias tan desiguales de la Tabla 4 y la forma de escalera de la acumulada en la Figura 5. Con esto vi que los parámetros del generador sí importan.

### 8.3. Python y Java (Tabla 7)

Los dos programas coincidieron en los 10 000 números de cada configuración y en todos los resultados de las pruebas. Creo que esto se debe a que los dos siguen la misma fórmula del LCG y los mismos pasos de la ficha, y a que los valores $p$ los calculé con las mismas fórmulas en ambos lenguajes. Con esto se cumple lo que esperaba la ficha: dos implementaciones que producen la misma secuencia con los mismos parámetros.

### 8.4. Qué se puede concluir y qué no

No rechazar $H_0$ quiere decir que, con esta muestra, no encontré evidencia suficiente para decir que los números no son uniformes. Pero eso no demuestra que el generador sea verdaderamente aleatorio: RANDU, por ejemplo, pasó las dos pruebas y aun así no es un buen generador. También hay que tener en cuenta que probé solo cuatro configuraciones con 10 000 números cada una, y que el valor $p$ de Kolmogorov-Smirnov que calculé es una aproximación, por lo que podría variar un poco frente a otras herramientas.

## 9. Preguntas de Control

1. **¿Qué es la hipótesis nula?**

   La hipótesis nula ($H_0$) es la suposición de partida que se somete a prueba. En esta práctica es que los números generados siguen una distribución uniforme $U(0, 1)$, es decir, que las diferencias entre lo observado y lo esperado se deben solo al azar [1].

2. **¿Qué representa el valor $p$?**

   Es la probabilidad de obtener una diferencia igual o mayor a la observada si $H_0$ fuera verdadera [1]. Si $p \geq 0{,}05$ no hay evidencia suficiente contra $H_0$; si $p < 0{,}05$, esa diferencia sería muy poco probable solo por azar.

3. **¿Qué significa rechazar $H_0$?**

   Significa concluir que hay evidencia en contra de $H_0$, porque el valor $p$ fue menor que $\alpha = 0{,}05$ (o el estadístico superó su valor crítico) [1]. En mi práctica ocurrió con la configuración $m = 16$, que solo genera 16 valores distintos.

4. **¿Qué diferencia existe entre Chi-cuadrado y KS?**

   Chi-cuadrado agrupa los datos en intervalos y compara las frecuencias observadas con las esperadas [1]. KS no agrupa: compara la distribución acumulada de los datos con la teórica y toma la mayor diferencia entre ellas.

5. **¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?**

Porque solo evalúa propiedades de una muestra finita; no rechazar $H_0$ indica que no hay evidencia en contra, no que el generador sea aleatorio [1]. Además, los generadores pseudoaleatorios son deterministas: con la misma semilla producen siempre la misma secuencia [2], y la de un LCG se repite al completar su período. En mi práctica, RANDU pasó las dos pruebas aunque es un mal generador.

# 10. Conclusiones

- Con esta práctica logré ver cómo un generador congruencial lineal produce números que parecen aleatorios y cómo se puede revisar su calidad con pruebas estadísticas, que era la idea central de la ficha. Además, programarlo en Python y en Java me ayudó a comprobar que ambos dan la misma secuencia con los mismos parámetros.
- Con la prueba Chi-cuadrado entendí que se revisa si los números se reparten de forma pareja en intervalos, y con Kolmogorov-Smirnov que se compara toda la distribución con la uniforme teórica. Aunque funcionan distinto, en mis resultados llegaron a la misma decisión en todas las configuraciones.
- Al comparar configuraciones me quedó claro que no cualquier parámetro sirve: las configuraciones base no se rechazaron, mientras que la de módulo pequeño se rechazó con mucha claridad.
- Interpretar el valor $p$ me hizo ver que no rechazar la hipótesis nula no significa que el generador sea perfecto, solo que no hay evidencia en contra con la muestra analizada. Por eso las pruebas ayudan a evaluar un generador, pero no alcanzan por sí solas para asegurar que sea aleatorio.

# 11. Declaración de uso de Inteligencia Artificial

En esta práctica utilicé dos herramientas de inteligencia artificial. Con **Claude** recibí ayuda para entender las fórmulas del generador, generar el código de las gráficas y de las pruebas Chi-cuadrado y Kolmogorov-Smirnov y para armar la estructura del proyecto en modelo, vista y controlador (que clases debería tener). Con **NotebookLM** obtuve las respuestas de las preguntas de control con sus referencias, a partir de las fuentes que cargué.

## 11.1. Prompts utilizados

**Prompt 1: Claude (comprender las fórmulas y la estructura).**

> *En la siguiente ficha ayudame a entender las fórmulas propuestas y generame un borrador con arquitectura Modelo Vista Controlador para tener una idea de que clases hacer. Debe ser en Python y Java, con clases de nombres fáciles.*

**Prompt 2: Claude (Gráficas).**

> *Ayudame haciendo el código para graficar lo dispuesto en la ficha, guiate de mi estructura y clases para obtener los datos mas relevantes y representarlos utilizando las tecnologías que propuse en el README*

**Prompt 3: NotebookLM (preguntas de control).**

> *Responde estas cinco preguntas de control usando únicamente las fuentes cargadas, en primera persona, con respuestas cortas y con las referencias en formato IEEE al final.*

## 11.2. Justificación del uso

**Herramientas utilizadas.**

- **Claude:** explicación de las fórmulas, código de las gráficas y estructura modelo-vista-controlador.
- **NotebookLM:** respuestas de las preguntas de control con sus referencias.

**Trabajo realizado por el estudiante.**

- Ejecutar los programas para revisar las gráficas, revisar el código, comprobar los resultados, creación del informe, verificar referencias de NotebookLM, etc.

# 12. Bibliografía

[1] J. S. Smith and D. T. Sturrock, *Simio and Simulation: Modeling, Analysis, Applications*. Sewickley, PA, USA: Simio LLC, 7th ed., August 2024.

[2] U. Wilensky, *The NetLogo 7.0.4 User Manual*. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, USA, 2025.

# 13. Anexos

## 13.1. Repositorio del proyecto

https://github.com/C-ael/Simulacion_Portafolio